Solving for LPPL Function Using Different Optimisation Algorithms

Optimisation Methods for Engineers Report

Jens Nielsen, Bogdan Dubinin

Abstract

The LPPL (Log-Periodic Power Law) function is used to predict the occurrence of bubbles in the stock market. An optimal solution of the function can predict the time at which a bubble is expected to burst, and the general outline of how the prices will grow.

The LPPL function is a non-convex function with many local minima, and therefore poses a non-trivial problem to fitting it against stock price time series. In this report, we test different optimisation methods and their efficiency in finding a suitable local minima for the LPPL function fit problem.

1 Introduction

The LPPL (Log-Periodic Power Law) function is a mathematical model used to predict the occurrence of critical events in complex systems, such as financial markets and earthquakes. This function is based on the idea that complex systems exhibit a self-similar behavior, which means that they tend to repeat patterns at different scales of time and space. The LPPL function is designed to identify these patterns and to estimate the probability of a critical event occurring in the near future.
The LPPL function is as follows:

$$f(x) = A - B(T-x)^m(1 + C\cos(\omega \ln(T-x) + \phi))$$

The function models a time series of prices so that $f(i) \approx p(i)$, where $p(1)...p(n)$ is our stock price time series. We have 7 unknown variables; so to fit this function against our time series, we have to find the optimal solution for a vector of length 7. If $m < 1$, LPPL grows super-exponentially until our critical time T - the time at which the bubble is expected to burst. If $C, \omega > 0$, LPPL exhibits oscillations increasing in frequency as we approach T.
In the following experiment, we use the Downhill-Simplex algorithm, the Particle Swarm Optimisation (PSO) algorithm and the Genetic algorithm (GA), along with 2 combinations of the PSO and GA algorithms, to determine which optimisation algorithm finds the best fit over the stock price time series for the S&P 500 bubble between 2003 and 2007. We additionally compare our fitted 7-length vectors against a reference optimal solution [1].

2 Choice of Method

For our implementation, we use the Downhil Simplex algorithm from the `scipy.optimize` package, and provide our own implementations of the GA and PSO algorithms. Due to LPPL's many local minima, determining the initial starting point is extremely important. Therefore,

we calculate an initial starting point $x_{initial}$ [1] given 3 points i, j, k in the time series, all consecutive in angle, meaning: $\omega \ln(T - i) = \omega \ln(T - j) + 2\pi, j > i$. In practice, this means choosing 3 peaks, or 3 troughs in the price graph. For the GA and PSO algorithms, each "person" of the population is assigned a random vector between an upper and lower bound. The bounds themselves are simple linear deviations from the initial starting point.
Even with an initial starting point, it is still a tricky problem. The Downhill-Simplex algorithm is expected to perform the worst, as it doesn't have a variable component to it, and will deliver the same result each time. The PSO and GA are expected to perform better, with the PSO algorithm having the most influential stochastic component, therefore more likely to find solutions the other 2 algorithms won't easily find. On the other hand, the GA algorithm finds the most optimal solution in a domain specified through the upper and lower bounds.
Lastly, we provide 2 "bonus" algorithms, by using the solution found by PSO as a starting point for GA, and by using the solution found by GA as a starting point for PSO. The hope is to have each of the algorithms strengths be applied to each other. As we see later, the order in which the algorithms are applied has an impact on the final solution. To measure the accuracy of the fit, we use the the mean-squared error of the fitted LPPL function and the stock prices as a loss function.

3 Algorithm Implementations

3.1 PSO

The PSO algorithm has free parameters w, a_{ind}, a_{grp} that determine the algorithm's performance. Through trial and error, we use for our final implementation the values $w = 1, a_{ind} = rand(n_p), a_{grp} = 0.15$, where $rand(n_p)$ is a random array of length n_p, number of particles, with values between 0 and 1. a_{grp} is crucial to the performance, as a too large value prevents the algorithm from converging to a solution.

3.2 GA

We implement the GA-algorithm in a standard fashion, with a few points worth mentioning. For the breeding mechanism, we take 2 parents and calculate the mean square to get the child. As for the mutation mechanism, we take predetermined lower and upper bounds and generate a "perturbation" vector. We then add this perturbation vector to our selected mutating vector.

4 Fitting Against S&P 500

We first fit against the S&P 500 time series from July 2003 to June 2007, whose actual peak was on 9.10.2007. We use a reference optimal solution [1]:

$$[A, B, T, m, C, \omega, \phi] = [7.47, 0.014, 1105.8, 0.52, -0.04, 9.87, 2.72]$$

Note that the critical time T=1105.8 (20.11.2007) overshoots by about a month. This is because the fitting algorithm doesn't derive the critical time, but rather generates and selects the best possible value.

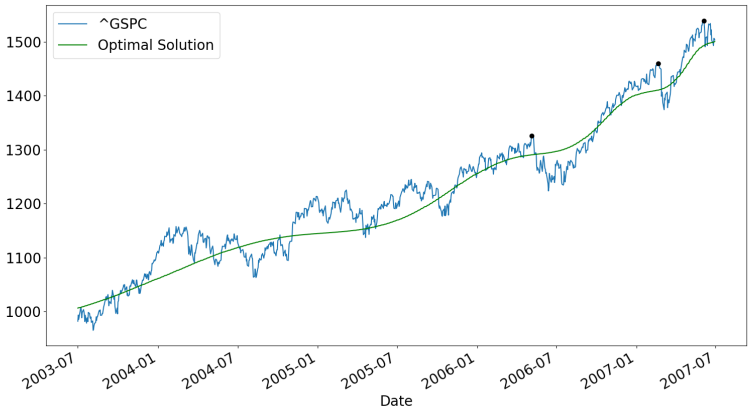

Figure 1: S&P 500 Optimal solution plot

The initial starting point for algorithms can be calculated [1] using 3 points consecutive in angle, in our case:

$$i = 5.5.2006 \qquad j = 2.20.2007 \qquad k = 4.6.2007$$

giving us

$$[A, B, T, m, C, \omega, \phi] = [9.46, 3.53 \cdot 10^{-4}, 1029.14, 0.5, 0, 6.21, 19.95]$$

Using each of our algorithms, we receive a plot for our optimal solutions, which are plotted in Figure 2. Both GA and Downhill perform poorly with no oscillation. Meanwhile, PSO shows the most similarities with the optimal solution, although the oscillations don't line up with the time series as tightly as the optimal solution.
As is show in the measurements below, the closest function in terms of smallest loss is the suprisingly GA function.

```
Optimal Loss is: 0.000741, Critical Time is: 1105.8 - 20.11.2007
Downhill loss is: 0.000869, Critical Time is: 1210.5 - 22.04.2008
GA loss is: 0.000769, Critical Time is: 1031.6 - 07.08.2007
PSO loss is: 0.000964, Critical Time is: 1067.8 - 27.09.2007
Actual critical time: 9.10.2007
```

The two-layer algorithms PSO-GA and GA-PSO algorithms show interesting properties in Figure 3. The PSO-GA algorithm, although started with the PSO solution given in Figure 2, finds a optimal solution extremely similar to the GA solution. On the other hand, the GA-PSO function finds a solution with a similar shape to the optimal solution, giving us a much better result.

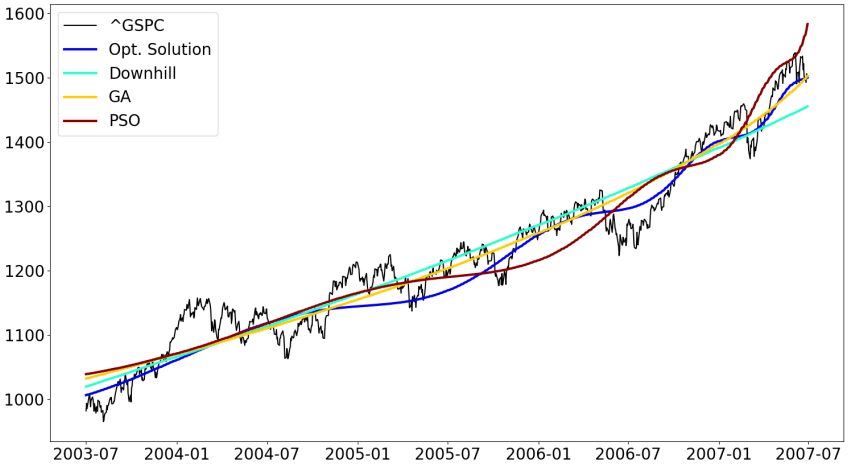

Figure 2: S&P 500, DS, GA, PSO

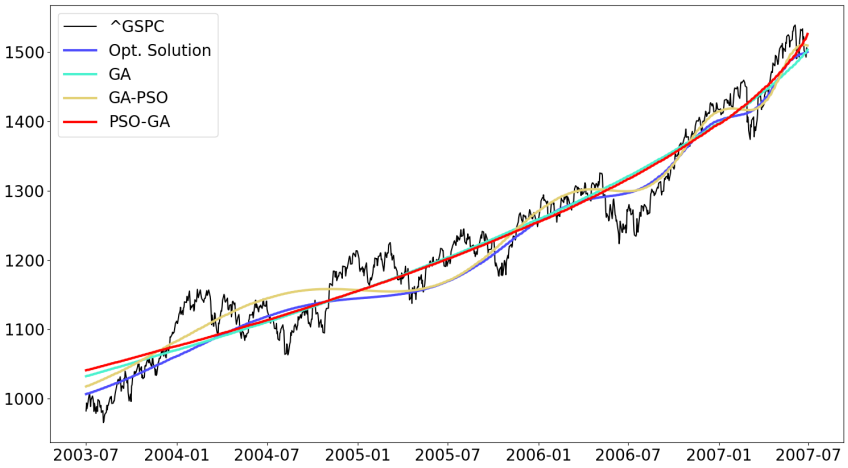

Figure 3: S&P 500, GA-PSO, PSO-GA

The critical time for both GA-PSO and the optimal reference solution are similar, and the GA-PSO loss is actually smaller than the optimal loss, indicating a tighter fit.

```
Optimal Loss is: 0.000741, Critical Time is: 1105.8 - 20.11.2007
GA-PSO Loss is: 0.000685, Critical Time is : 1108.2 - 22.11.2007
PSO-GA Loss is:  0.00076, Critical Time is : 1016 - 16.7.2007
Actual critical time: 9.10.2007
```

Multiple runs of the algorithms provide the same results, with the GA-PSO algorithm consistently providing the best fit.

5 Conclusion

The solutions given by the different algorithms is telling of all their strengths and weaknesses. Fitting against the time series provides a challenge, since the algorithms are inclined to prefer a fit more similar to a linear or parabolic function. We see this in Figure 2, in which the GA loss is the smallest amongst the given algorithms.
For the S&P500 in Figure 2 and 3, we see that the PSO-GA, although taking the PSO solution as an initial point, still finds a function similar to a parabolic fit, similar to the GA solution - an indication that the implemented GA algorithm gravitates towards smaller oscillations. This is supported by the fact that the GA and PSO-GA solutions both have C parameters close to 0. A possibility not given in this report would be to alter the loss-function to give certain parameters priority. The Downhill-Simplex algorithm performs poorly as expected, and while the PSO gives a solution with a good shape, numerically it performs poorly, and has an above average loss.
The GA-PSO algorithm consistently performs the best in our experiment runs, both numerically in terms of loss, and how it visually fits to the time series. One possible reason for why the PSO algorithm is more inclined to find larger oscillating solutions is because it has a strong stochastic component, therefore being able to more easily escape local minima with small oscillating properties. A possible future experiment to confirm this would be to test algorithms with strong stochastic components, such as Simulated Annealing algorithm.

References

[1] Vincenzo Liberatore. Computational lppl fit to financial bubbles, 2011.